Final Project Proposal: SSH Tunneling System

Hüseyin Kaya

Student ID: 220303036

Date: 22/01/2026

Abstract

This project presents the final implementation of a secure SSH-style tunneling system developed in the C programming language. The system establishes a secure channel between a client and a server using a hybrid cryptographic approach. The connection initiates with an **RSA-based handshake** to securely exchange a session key, followed by **AES-128 (ECB mode)** symmetric encryption for all subsequent data transfer.

The implementation features a robust **Stop-and-Wait ARQ** (Automatic Repeat Request) mechanism. Files and messages are segmented into frames protected by **CRC32** checksums. To realistically test the protocol's reliability, a separate "Middlebox" (Proxy) software has been developed to inject random noise and corrupt frames during transmission. The server detects these corruptions via CRC checks and requests retransmission (NACK), ensuring data integrity. The final system successfully demonstrates secure key exchange, encrypted file transfer, and error recovery over TCP sockets.

Arel Üniversitesi, Müh. Mim. Fak. Bilgisayar Mühenendisliği. Türkoba Mah., Erguvan Sok., No:26, K 34537, Tepekent – Büyükçekmece, Istanbul

1 Introduction

In this finalized project, a secure communication tunnel has been implemented that mimics the fundamental behavior of the SSH protocol. The system is built upon **TCP socket programming** and operates in a client-server architecture, supported by an intermediate proxy for testing network reliability.

The core functionality begins with a cryptographic handshake . The server generates an RSA public/private key pair and shares the public parameters (n, e) with the client. The client then generates a random 128-bit AES session key, encrypts it with the server's RSA public key, and transmits it. Once the secure session is established, all payloads—text messages and binary files—are encrypted using **AES-128 in Electronic Codebook (ECB) mode**.

A critical feature of this implementation is the **Reliable Data Transfer** layer. Data is transmitted in discrete frames containing a sequence ID, payload size, and a CRC32 checksum. A "Middlebox" application acts as a tunnel between the client and server, randomly flipping bits to simulate network noise. The receiver verifies the integrity of each frame; if a frame is corrupted, a NACK (Negative Acknowledgment) is sent, triggering an automatic retransmission. This ensures that the reconstructed file at the destination is identical to the source.

2 System Design and Diagrams

The system architecture is designed to simulate a real-world insecure network environment where secure data transmission is required. The design is modular, separating the application logic (file transfer) from the security protocols (RSA/AES) and the network transport layer.

2.1 Architectural Components

The implemented system consists of three distinct entities operating synchronously:

- **The Client Node:** This component initiates the connection, reads local files, applies AES-128 encryption, and encapsulates data into frames. It manages the transmission loop, strictly waiting for positive acknowledgments (ACK) before proceeding to the next frame.
- **The Middlebox (Proxy):** A crucial addition to this final implementation, this component sits between the client and server. It acts as a transparent TCP relay but includes a probabilistic noise generator to corrupt passing frames, thereby testing the system's error resilience.
- **The Server Node:** Acting as the listener, it maintains the RSA private key for the initial handshake and an AES session key for the duration of the connection. It validates CRC checksums, sends ACK/NACK responses, and manages file reconstruction on the disk.

2.2 Diagrammatic Representations

Two visual models have been created to depict the static structure and dynamic behavior of the system.

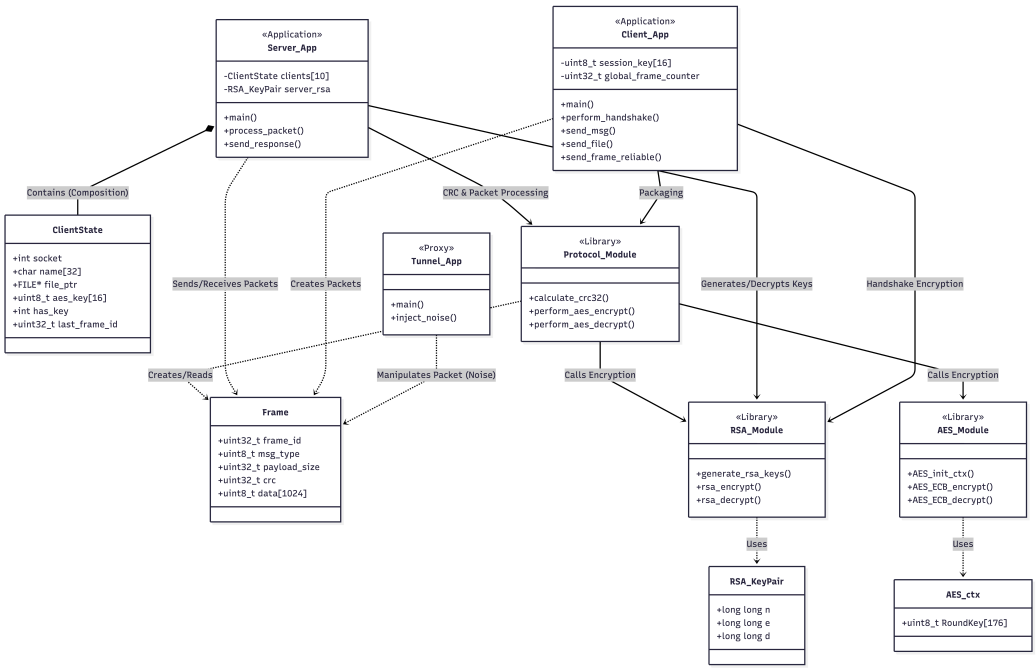

Figure 1: UML Class Diagram of the Implemented System

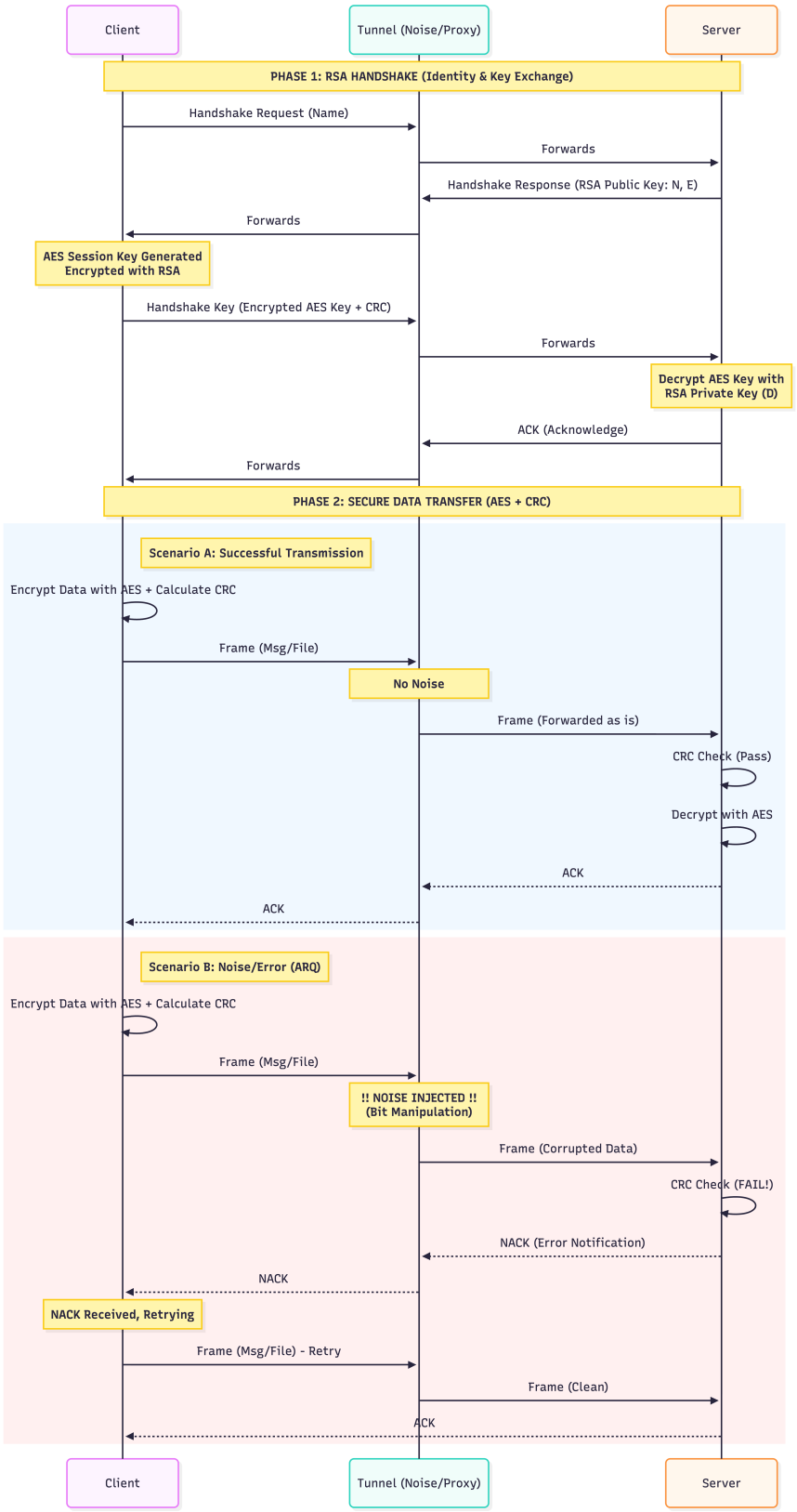

Figure 2: Sequence Diagram: Handshake, Noise Injection, and ARQ Recovery

The **UML Class Diagram** (Figure 1) illustrates the modular design, highlighting the separation between the Client, Server, and the noise-injecting Middlebox. It also depicts the dependencies on helper modules for cryptographic operations and the protocol structure.

The **Sequence Diagram** (Figure 2) details the chronological communication process . It demonstrates the initial RSA handshake for key exchange, followed by the "Stop-and-Wait" ARQ loop. Specifically, it shows how the Client sends encrypted frames through the Middlebox, how the Server detects corruption (NACK), and how retransmission ensures reliable delivery.

References

1. William Stallings, *Data and Computer Communications*, 10th ed., Pearson, 2014.

2. Daniel J. Barrett and Richard E. Silverman, *SSH, The Secure Shell: The Definitive Guide*, O'Reilly Media, 2005.

3. SSH Academy. (n.d.). *SSH Tunneling Explained*. Retrieved from `https://www.ssh.com/academy/ssh/tunneling`

4. Kurose, J. F., & Ross, K. W. (2017). *Computer Networking: A Top-Down Approach*. Pearson.